RICE UNIVERSITY

Framework and GPU Interaction

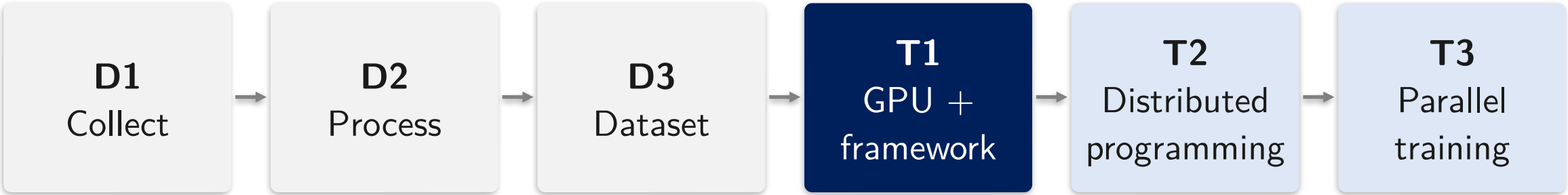
Fusion, memory, input pipelines, and precision: what the framework does around your kernels

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 10, Lecture 1

Where we are in the pipeline



Week 9: tensors, dispatch, the forward pass; graphs and torch.compile

In our pipeline

Today: what the framework does **around** your kernels. **T1 due Friday**; Lab 3 due after this lecture.

Lecture 2 this week starts distributed programming: T2 and Lab 4 are released after it.

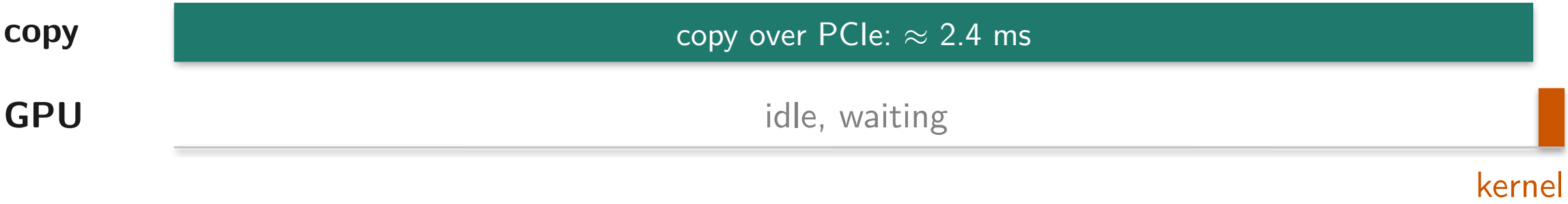
Why isn't a fast kernel enough?

A training batch of images in FP32 sits in CPU memory. Your kernel is fast:

Kernel reads the batch
from GPU memory: $\approx$ 0.05 ms

Copy the batch first
CPU $\rightarrow$ GPU over PCIe, every step

How busy is the GPU if each step waits for its copy?



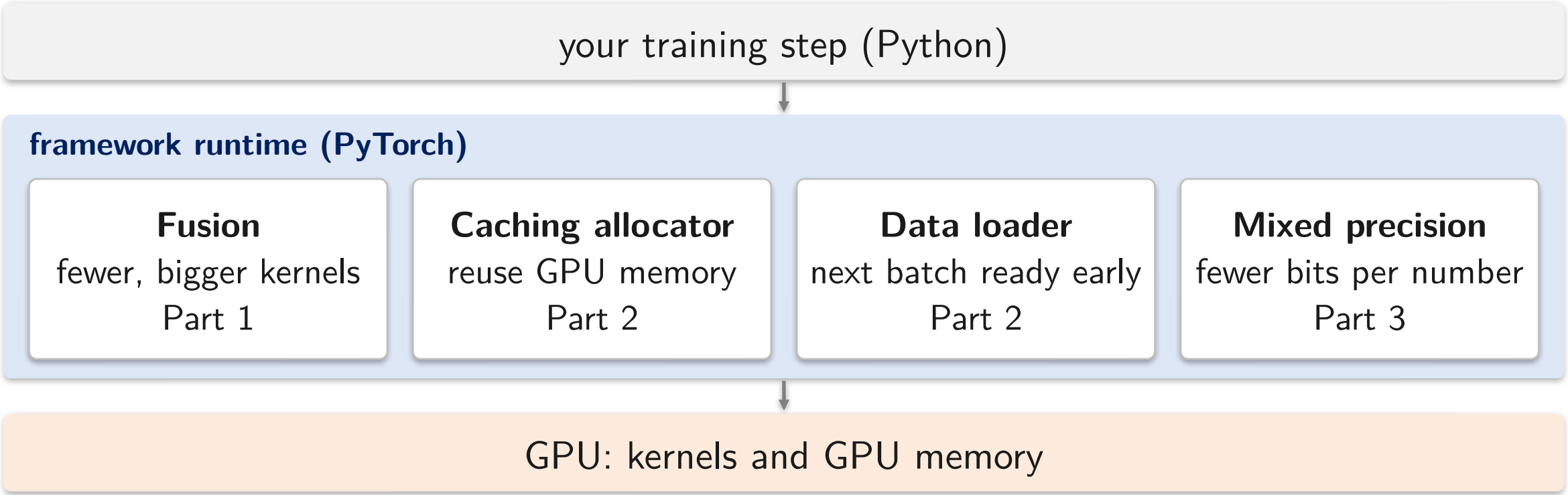
Watch out

The GPU is busy about **2%** of the step. A faster kernel cannot fix that; the framework must get data and memory ready **ahead** of the kernels.

Source: PCI-SIG, PCIe 5.0 x16 $\approx$ 64 GB/s per direction (theoretical); H100 datasheet (HBM). Batch is illustrative; times derived.

What the framework adds around kernels

runtime: the code inside PyTorch that decides how and when your step runs on the GPU.



Each piece removes one way the GPU **waits**: on launches, on memory, on data, on bytes.

Source: PyTorch docs, "CUDA semantics" and torch.utils.data; Ansel et al., "PyTorch 2," ASPLOS 2024.

Where these show up in PyTorch

Part	In PyTorch	What it costs you
1 Fusion	torch.compile (Inductor)	compile time
2 Memory and input	caching allocator, DataLoader	held memory, CPU cores
3 Precision	torch.amp (autocast, BF16)	rounding, overflow risk

T1, last part (today's BUILD, Lab 3): integrate your Triton kernel, then **test one bottleneck hypothesis** with the profiler.

Why we need it

A fast kernel only helps if it is **fed**. The framework fuses work, reuses memory, stages data ahead, and shrinks numbers so the GPU does not wait.

Source: PyTorch docs: torch.compile, "CUDA semantics," DataLoader, torch.amp; Micikevicius et al., ICLR 2018.

Learning goals for today

- **Predict** the bytes and launches saved when the compiler **fuses** operations.
- **Explain** how PyTorch reuses GPU memory, and how a batch reaches the GPU **without making it wait**.
- **Compare** FP32, BF16 and FP16, and explain **mixed precision**.
- **Integrate** a forward-only Triton kernel and **test** one bottleneck hypothesis.

A puzzle to start

Profiler, one training step
wall-clock time: 100 ms
GPU kernels busy: 30 ms

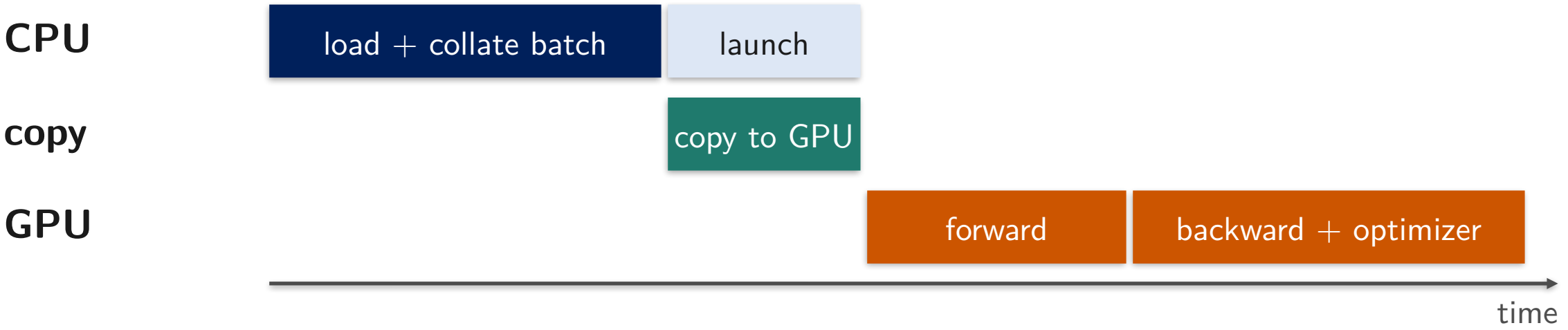
The upgrade
a new GPU that runs
every kernel 2× faster

How long does one step take after the upgrade?

Answer

70 + 30 / 2 = **85 ms**: only 15% less time. The rest is launches, copies, and waiting — today's subject.

Anatomy of one training step



Today's plan

Part 1 shortens the **GPU** lane. Part 2 hides the **CPU and copy** lanes behind it. Part 3 makes every byte and FLOP **cheaper**.

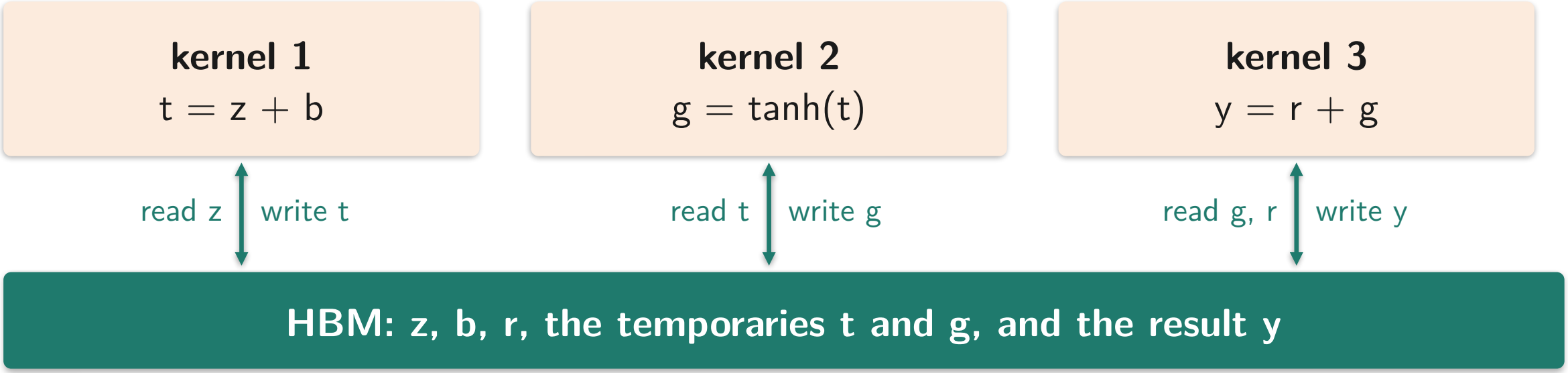
Part 1

Fusion

What does the compiler fuse, and how much does it save?

Eager mode: one kernel per operation

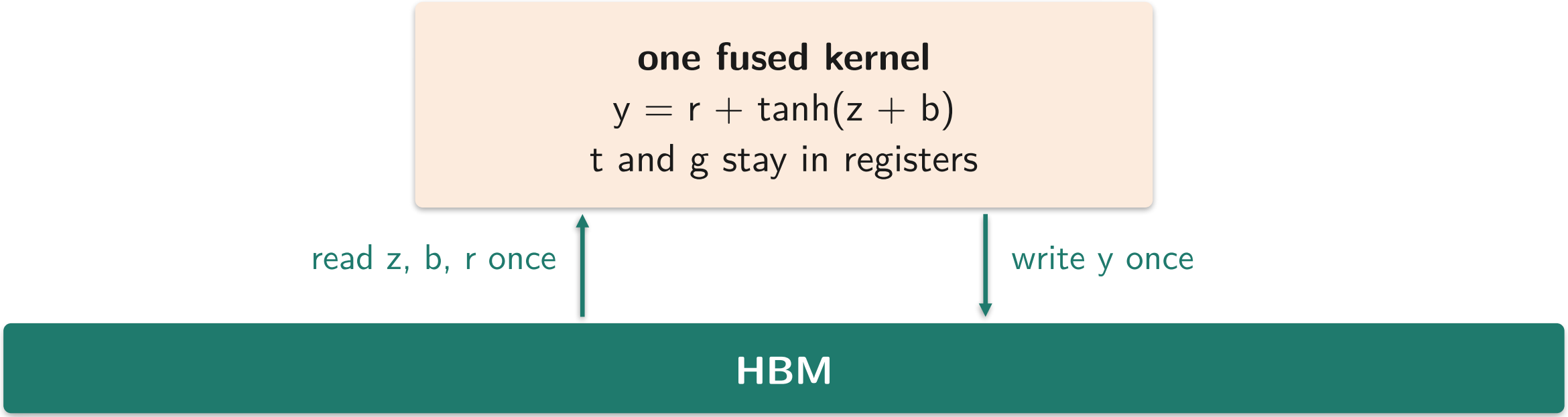
kernel: one program the GPU runs · HBM: the GPU's main memory



Watch out

Every temporary makes a **round trip to HBM**, and every operator pays a **launch**.

Fused: one kernel for the whole chain



Key idea

Fusion keeps the arithmetic and drops the **temporaries** and the extra **launches**.

Bytes saved: count the passes

pass = one full read or write of a large tensor in HBM

$$\text{unfused: } (2 + 2 + 3) \text{ passes} = 7$$

$$\text{fused: read } z, r \ + \ \text{write } y = 3 \text{ passes}$$

$$\text{memory-bound: time} \propto \text{passes} \ \Rightarrow \ \text{speedup} \approx 7 \ / \ 3 \approx 2.3\times$$

Memory

For **memory-bound** code (time set by bytes, not arithmetic): speedup $\approx$ passes before $\div$ passes after.

What torch.compile fuses

Code	Fused by torch.compile?
A chain of elementwise operations	**yes**: one kernel
A matrix multiply	no: a call to NVIDIA's library
Code on both sides of a graph break	no: two separate graphs

Key idea

The compiler fuses the **memory-bound glue** between matrix multiplies — exactly where passes are wasted.

Source: PyTorch docs, torch.compile and TorchInductor; Ansel et al., "PyTorch 2," ASPLOS 2024.

Fusion in our MLP's hidden layer

eager

bias kernel + tanh kernel

4 passes · 2 launches

fused (your T1 kernel)

$h = \tanh(z + b)$

2 passes · 1 launch

$$\text{bytes saved} \approx 33 \text{ KB} \rightarrow \approx 10 \text{ ns}$$

$$\text{one launch saved} \rightarrow \approx 4 \ \mu s$$

Memory

At $B = 64$, fusion pays through **launches**, not bytes.

Source: H100 datasheet (HBM 3.35 TB/s); ≈ 4 µs per launch is the Week 8 round assumption.

Check yourself

Two minutes with a neighbour.

- Three elementwise kernels, each one input and one output. Passes before and after fusion? Speedup?
- Would fusing a bias add into a large matrix multiply speed it up much? Why?
- For our MLP at $B = 64$, what does fusing bias + tanh mainly save?

Part 2

Memory and getting data in

Where does a tensor's memory come from, and how does a batch reach the GPU without making it wait?

PyTorch's caching allocator

Getting memory from the GPU driver is slow, and freeing it **stalls the CPU**.

GPU memory · PyTorch asks the driver for a few large segments and keeps them



Key idea

A freed tensor's block goes back to **PyTorch's pool**, not the driver; the next tensor **reuses** it with no waiting.

Source: PyTorch docs, "CUDA semantics": memory management; CUDA C++ Programming Guide, implicit synchronization.

Allocated versus reserved

```
x = torch.empty(2**26, device="cuda")  # 256 MiB
MB = 2**20
torch.cuda.memory_allocated() // MB          # 256
torch.cuda.memory_reserved() // MB           # 256
del x
torch.cuda.memory_allocated() // MB          # 0
torch.cuda.memory_reserved() // MB           # 256
```

- **allocated**: bytes in live tensors.
- **reserved**: bytes PyTorch keeps cached.

Key idea

nvidia-smi shows **reserved** memory (plus overhead), so it always looks bigger than your tensors.

Source: PyTorch docs, "CUDA semantics" (memory management, pinned memory, streams).

Fragmentation



10 GB reserved, 0.5 GB per cell · teal: live tensors · white: free

free: 6 GB in all, largest free run: 1.5 GB

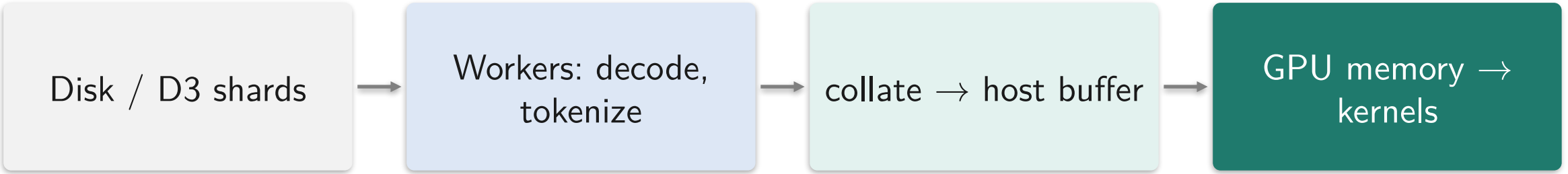
`a 2 GB tensor → CUDA out of memory`

Remedies

Fix: keep tensor **shapes consistent** from step to step and drop references you no longer need.

Source: PyTorch docs, "CUDA semantics": optimizing memory usage.

The path of one batch



PCIe: the CPU–GPU link

$$\text{HBM / PCIe} \approx 3.35\ \text{TB/s} / 64\ \text{GB/s} \approx 52$$

Key idea

Cross PCIe **once**, then keep data on the GPU; prepare the next batch **while** the GPU computes.

Source: PCI-SIG, PCIe 5.0 x16 ≈ 64 GB/s per direction (theoretical); H100 datasheet (HBM).

DataLoader: workers prepare batches ahead

```
loader = DataLoader(dataset,
                    batch_size=64,
                    shuffle=True,
                    num_workers=4,     # processes
                    pin_memory=True)   # next slide
for x, y in loader:
    x = x.to("cuda", non_blocking=True)
    train_step(x, y)
```

- Workers build the **next** batches while the GPU trains.
- Add workers until the GPU **stops waiting**.

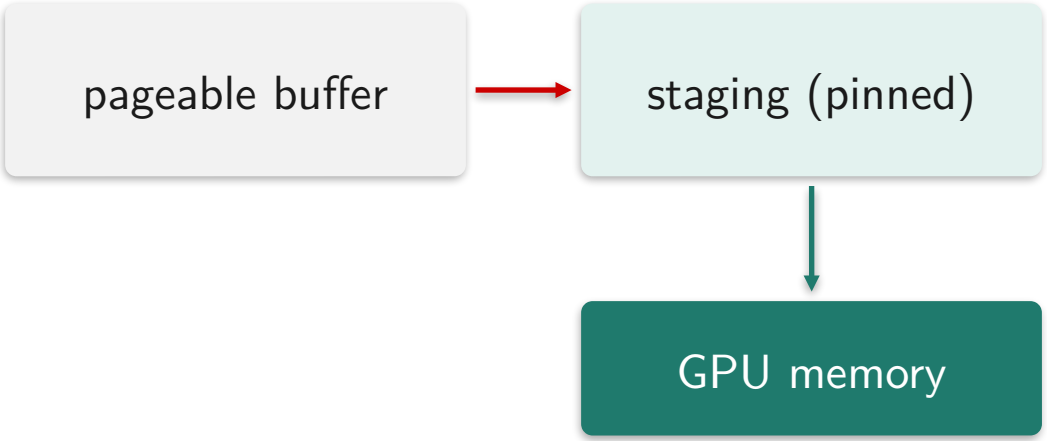
Watch out

With **0 workers** the main process loads each batch between steps, and the GPU sits idle.

Source: PyTorch docs, torch.utils.data.DataLoader.

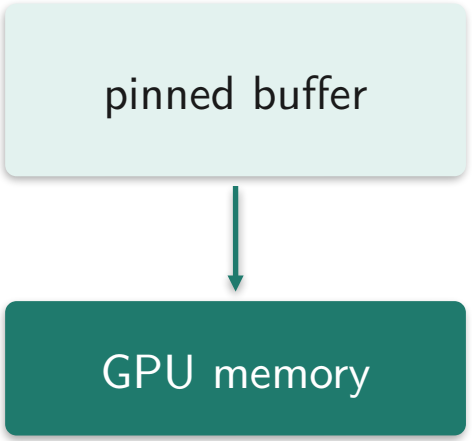
Pageable versus pinned memory

Pageable (ordinary)



the CPU copies first; the call waits

Pinned (fixed in place)



the GPU reads it directly;
the CPU moves on at once

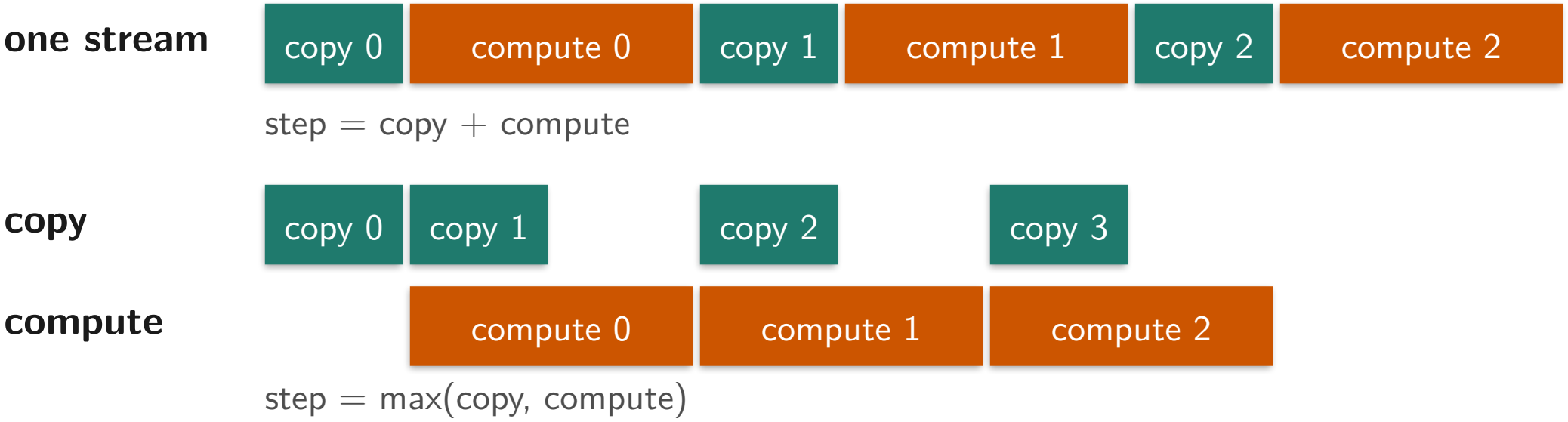
Watch out

An asynchronous copy (`non_blocking=True`) works **only** from pinned memory. Pin the next batches, not the whole dataset.

Source: PyTorch docs, "CUDA semantics": pinned memory buffers; CUDA C++ Programming Guide.

Overlapping copies with compute

stream: a queue of GPU work that runs in order; two streams can run at the same time



Key idea

Overlapped: step = **max**(load, copy, compute), not the sum.

Source: PyTorch docs, "CUDA semantics" (memory management, pinned memory, streams).

Hidden synchronizations

synchronization: the CPU stops and waits for the GPU to finish its queue

Code	Why the CPU must wait
`loss.item()`, `print(loss)`	needs the value on the CPU
`t.cpu()`, `t.tolist()`	copies a result back to the CPU
`if loss > 10:`	a Python branch on a GPU value

Watch out

Log every N steps, or keep running sums **on the GPU** and read them once.

Source: PyTorch docs, "CUDA semantics: asynchronous execution".

Check yourself

Two minutes with a neighbour.

- You `del` a 1 GB tensor. What happens to allocated and to reserved memory?
- The GPU idles at the start of every step, and `num_workers=0`. What is happening?
- Load 5 ms, copy 2 ms, compute 8 ms per step: step time serial vs overlapped?

Part 3

Precision

How few bits can we compute with, and what breaks when we try?

Anatomy of a floating-point number

A number is stored as **sign** $\times$ **(1 + fraction)** $\times$ **a power of 2**.

FP32



- **Exponent** bits $\rightarrow$ **range**: how large or small a number can be.
- **Mantissa** bits $\rightarrow$ **precision**: the gap after 1 is $\varepsilon = 2^{-23} \approx 1.2 \times 10^{-7}$.

Source: IEEE 754-2019 (FP32, FP16); NVIDIA A100 and H100 whitepapers (BF16).

Three formats, bit by bit

grey: sign · teal: exponent bits · orange: mantissa bits · one cell = one bit



Newer GPUs also compute in 8-bit **FP8**: even fewer bits, with extra scaling.

Key idea

BF16 keeps FP32's **range** and gives up **precision**; FP16 does the opposite.

Source: IEEE 754-2019 (FP32, FP16); NVIDIA A100 and H100 whitepapers (BF16).

Why fewer bits are faster

Memory-bound kernel
2 bytes instead of 4
$\approx$ **2×** faster

Matrix multiply
FP32 cores: 67 TFLOP/s
BF16 tensor cores: **989** TFLOP/s

tensor core: a unit in the GPU that multiplies small matrices in low precision

Key idea

Fewer bits: fewer **bytes** and much faster **matrix multiplies** — paid for in range or precision.

Source: NVIDIA H100 Tensor Core GPU datasheet (SXM5); NVIDIA H100 Tensor Core GPU Architecture whitepaper (2022). Dense tensor-core values.

Two ways low precision fails

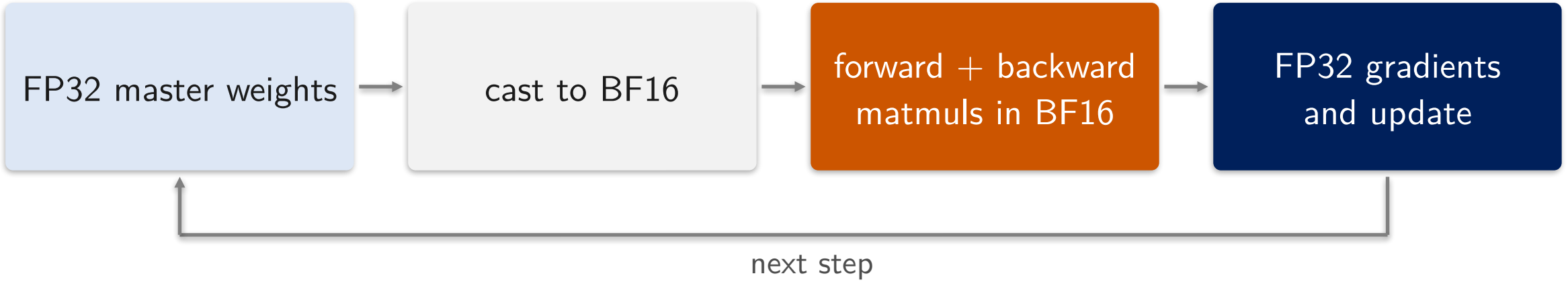
Underflow (range)
gradient g $= 10^{-8}$
FP16: rounds to **0**
BF16, FP32: kept

Swamping (precision)
w + update $= 1 + 10^{-4}$
BF16, FP16: $=$ **1** (update lost)
FP32: kept

Check

Fixes: keep the weights and the update in **FP32** (swamping); **scale the loss** in FP16 (underflow).

Mixed precision: the recipe



Key idea

Matrix multiplies in **BF16** for speed; softmax, sums, the loss, and the update in **FP32** for accuracy.

Source: Micikevicius et al., "Mixed Precision Training," ICLR 2018; PyTorch AMP docs.

Mixed precision in PyTorch

```
model = MLP().cuda()              # FP32 weights
for x, y in loader:
    with torch.autocast("cuda",
                        dtype=torch.bfloat16):
        loss = F.cross_entropy(model(x), y)
    loss.backward()                       # FP32 .grad
    opt.step()                            # FP32 update
    opt.zero_grad()
```

- Wrap the **forward and loss** only.
- Parameters and gradients stay **FP32**.

Key idea

Autocast picks the format **per operation**: matrix multiplies go low, sums and the loss stay FP32.

Source: PyTorch docs, "Automatic Mixed Precision" (torch.amp); Micikevicius et al., ICLR 2018.

FP16 needs loss scaling; BF16 does not

$$\text{scaled loss: } S \cdot L, \quad e.g.\ S = 2^{16}$$

$$\text{chain rule: } \partial(S \cdot L) \,/\, \partial w = S \cdot \partial L \,/\, \partial w$$

$$10^{-8} \times 2^{16} \approx 6.6 \times 10^{-4} \quad \text{fits in FP16}$$

$$\text{before the update: divide by } S \text{ (in FP32)}$$

Check

BF16 has FP32's exponent range: gradients do not underflow, so it needs **no scaling**.

Source: PyTorch docs, "Automatic Mixed Precision" (torch.amp); Micikevicius et al., ICLR 2018.

Check yourself

Two minutes with a neighbour.

- An activation reaches 70,000. Which of FP16 and BF16 can store it? Why?
- Weights kept in BF16, $w = 1$, update 10^{-3}. What happens? How does mixed precision avoid it?
- Why does BF16 training need no loss scaling?

IN-CLASS BUILD

Integrate and profile

Pairs: put your T1 kernel in the forward pass, then predict and measure.

Step 1: wrap the kernel as an operator

```
@triton_op("t1::bias_tanh", mutates_args={})
def bias_tanh(z: Tensor, b: Tensor) -> Tensor:
    h = torch.empty_like(z)
    n, H = z.numel(), z.shape[-1]
    grid = (triton.cdiv(n, 1024),)
    wrap_triton(bias_tanh_kernel)[grid](
        z, b, h, n, H, BLOCK=1024)
    return h
```

- A **named operator** wraps your kernel.
- In the forward: `bias_tanh(x @ W1, b1)`.

In our pipeline

No backward: autograd is a provided tool, so your kernel runs in **forward-only** passes; training keeps PyTorch's layers.

Source: PyTorch docs, torch.library: `triton_op` and `wrap_triton` (PyTorch 2.6+).

Step 2: validate the forward

```
SHAPES = [(64, 64), (1, 64), (999, 64), (5, 33)]
for B, H in SHAPES:
    for dt in (torch.float32, torch.bfloat16):
        kw = dict(device="cuda", dtype=dt)
        z = torch.randn(B, H, **kw)
        b = torch.randn(H, **kw)
        assert_close(bias_tanh(z, b),
                     torch.tanh(z + b))
```

- Odd shapes: **one row**, a size that is **not a multiple** of the block, an **odd width**.
- Both **FP32** and **BF16**.

Check

Then time the **forward pass only**, with autograd switched off — eager vs compiled, with and without your kernel.

Source: PyTorch docs, torch.testing: `assert_close`; torch: `inference_mode`.

Step 3: test a hypothesis, not a hunch

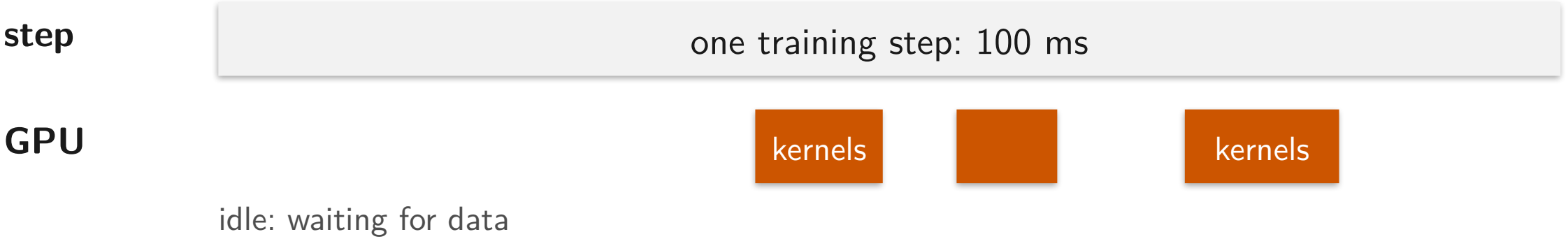
Hypothesis	Prediction	Measurement	Conclusion
"The step is launch-bound."	GPU busy $< 50\%$; batch $\times 2$: same time	profiler + one intervention	accept or reject; next test

launch-bound: time set by the CPU starting many small kernels, not by their work

Key idea

A profile shows **where** time goes; an **intervention** that changes one thing tests **why**.

Step 4: measure the GPU busy fraction



$$\text{GPU busy} = \text{kernel time} \;/\; \text{step time} = 30 \;/\; 100 = 30\%$$

Check

The profiler gives both numbers. **Skip the first steps**, so compile time is not counted.

Source: PyTorch recipe "PyTorch Profiler"; torch.profiler documentation.

Exercise: predict before you profile

Hypothesis	GPU busy	Batch × 2	Cached batch
H1: launch-bound			
H2: data loading starves the GPU			
H3: kernel-bound (control)			

USE

Batch 64, our T1 MLP. Cached batch: one batch kept on the GPU, reused every step.

Fill in: low / high, $\approx$ same / $\approx 2\times$, faster / no change.

STRETCH

Which single intervention best separates H1 from H2?

Exercise: solutions

Hypothesis	**GPU busy**	**Batch × 2**	**Cached batch**
H1: launch-bound	low, gaps	$\approx$ same time	no change
H2: data starves the GPU	low, idle at start	$\approx 2\times$ (loading)	**much faster**
H3: kernel-bound	$\approx 100\%$	$\approx 2\times$	no change

In our pipeline

The **cached-batch** intervention separates H1 from H2. For the T1 MLP expect **H1** — Week 8's overhead-bound prediction.

Key takeaways

- **Fusion** keeps temporaries on chip: speedup $\approx$ passes before $\div$ after; at our size it saves **launches**.
- PyTorch **caches** GPU memory: reserved $\geq$ allocated; fragmentation can cause OOM.
- **Workers + pinned memory + streams**: step time $=$ max(load, copy, compute), not the sum.
- **Exponent bits = range, mantissa bits = precision**; mixed precision: BF16 compute, FP32 update.
- Put the kernel in the **forward pass**, validate it, and test hypotheses by **intervention**.

Next time, and readings

When	What
Friday (this week)	T1 due: kernel, integration, compile vs eager, profiling analysis
After this lecture	Lab 3 due
Lecture 2 (this week)	Distributed programming; T2 and Lab 4 released after it

READINGS

PyTorch docs: "CUDA semantics" and "Automatic Mixed Precision".

Micikevicius et al., "Mixed Precision Training," ICLR 2018.